

# PLC Programming Languages and Software Architecture

From Ladder and Structured Text to state machines, reusable modules and testable control code

IEC 61131-3 | LD | ST | FBD | SFC | ownership | state | libraries | testing | Git

|                        |                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Audience</b>        | Mechanical, process, manufacturing, instrumentation and automation engineers                                          |
| <b>Target duration</b> | 42-46 minutes                                                                                                         |
| <b>Format</b>          | Yael - young mechanical engineer; Amir - senior process and controls engineer                                         |
| <b>Core example</b>    | A CIP skid starts a pump against a closed path after power recovery because state and command ownership are ambiguous |
| <b>Deliverable</b>     | Full script + NotebookLM prompt + language matrix + architecture map + engineering checklist + sources                |

*Educational material. It does not replace a functional specification, hazard and risk assessment, validated software lifecycle, vendor documentation, cybersecurity controls or the contractually applicable editions of standards.*

## 1. Pre-production alignment check

- The episode continues directly from Episode 6: a deterministic network can still deliver a poorly structured command exactly on time.
- It combines the master plan topics of PLC languages, code management and software quality into one architectural foundation; later episodes apply the foundation to HMI, data, safety and cybersecurity.
- The audience remains mechanical and process engineers. Syntax is explained only where it reveals physical behavior, timing, ownership or maintainability.
- No language is ranked as universally best. The selection criterion is how clearly it expresses and verifies the engineering intent.
- The ending creates a direct bridge to Episode 8 on HMI hierarchy, operator commands and alarm presentation.

**Freshness check** | The standards map was checked against current official IEC, PLCopen, ISA/OMAC, CODESYS, Siemens and Rockwell Automation material. It reflects IEC 61131-3:2025 edition 4, IEC TR 61131-8:2017, IEC 61131-10:2019 and PLCopen software-construction guidance including the 2023 quality-metrics guideline. Tool features and file formats remain vendor- and version-specific; verify the project platform and adopted editions.

## 2. Episode objectives

- Turn functional requirements into explicit modes, states, commands, permissives, interlocks, trips, invariants and restart rules.
- Explain the current IEC 61131-3 language suite and the roles of programs, function blocks and functions.
- Choose Ladder Diagram, Structured Text, Function Block Diagram and SFC/state-machine structures by engineering purpose.
- Design a layered control application with one owner for each physical output and typed module interfaces.
- Separate configuration, recipe data, live state, retained evidence and restart decisions.
- Use reusable libraries and object-oriented features without hiding behavior or destroying online diagnosability.
- Create a practical lifecycle using version control, reviews, static analysis, simulation, tests, release evidence and verified restore.

## 3. Timing and segment plan

| Time        | Segment                  | Purpose                                                                  |
|-------------|--------------------------|--------------------------------------------------------------------------|
| 00:00-03:30 | Cold open                | A retained sequence state starts a pump before the valve path is proven. |
| 03:30-07:30 | Requirements before code | Define behavior, invariants, restart and protection vocabulary.          |
| 07:30-12:30 | IEC 61131-3 landscape    | Languages, POUs, exchange formats and portability limits.                |
| 12:30-17:30 | Ladder Diagram           | Transparent Boolean decisions and common ownership traps.                |
| 17:30-22:30 | Structured Text          | Algorithms, data handling, bounded execution and explicit state.         |
| 22:30-26:30 | Function Block Diagram   | Signal flow, basic control and execution-order discipline.               |
| 26:30-31:30 | SFC and state machines   | Explicit steps, transitions, abnormal exits and PackML concepts.         |
| 31:30-36:30 | Layered architecture     | I/O, device, equipment, sequence and supervisory boundaries.             |
| 36:30-40:30 | Data and reuse           | Typed models, library versions, interfaces and restrained OOP.           |
| 40:30-44:00 | Quality workflow         | Version control, static analysis, simulation and test hierarchy.         |
| 44:00-46:00 | Resolution and close     | Rebuild ownership and recovery; bridge to HMI.                           |

## 4. Maintainable control-software architecture

| Layer                   | Responsibility                                                                        | Contract / evidence                                                                 |
|-------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1. I/O abstraction      | Channel mapping, scaling, inversion, signal quality, forcing and simulation boundary. | Engineering units, quality status, channel diagnostics and controlled substitution. |
| 2. Device module        | One pump, valve, heater, drive or instrument; owns the physical command.              | Typed command/status, permissives, interlocks, feedback, fault reason and timing.   |
| 3. Equipment module     | Coordinates devices to provide circulation, dosing, heating, positioning or transfer. | Capability request, readiness, completion, hold, abort and fault behavior.          |
| 4. Sequence / phase     | Executes the recipe or machine cycle through explicit states and transitions.         | Entry criteria, invariants, completion criteria, timeout and abnormal exits.        |
| 5. Supervisory boundary | Modes, production orders, handshakes, recipe approval and HMI-facing summaries.       | Authority, audit trail, command confirmation and compact situational status.        |
| Cross-cutting lifecycle | Requirements, naming, libraries, version control, tests, release and restore.         | Traceability from change request to source, binary, target, test report and backup. |

## 5. Language and structure decision matrix

| Language / structure         | Strong fit                                                                             | Review questions                                                                                   |
|------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Ladder Diagram (LD)          | Boolean decisions, permissives, interlocks, mode selection and simple device control.  | Is there one writer? Are latches, priority, reset and scan dependencies explicit?                  |
| Structured Text (ST)         | Calculations, arrays, recipe validation, algorithms, bounded loops and state machines. | Are types, units, execution bounds, array limits and side effects controlled?                      |
| Function Block Diagram (FBD) | Signal flow, PID, filtering, scaling, voting and analog basic control.                 | Does each page tell one signal story? Is execution order understood and each block contract clear? |
| SFC / explicit state machine | Sequential behavior, phases, legal transitions, hold, abort and recovery.              | Are states meaningful, transitions deterministic and abnormal exits complete?                      |
| Function                     | Stateless calculation with defined inputs and result.                                  | Is it deterministic and free of hidden persistent state or output side effects?                    |
| Function block               | Reusable behavior with instance memory, such as a valve, motor or controller.          | Is ownership clear, internal state encapsulated and the interface small and testable?              |
| Program / orchestration      | Coordinates application modules under a scheduled task.                                | Does it coordinate requests rather than bypassing device ownership?                                |

**Core principle** | Choose the representation that makes the required physical behavior, state, ownership, timing and failure response easiest to review and test. Familiar syntax cannot rescue ambiguous

architecture.

## 6. NotebookLM production prompt

**Paste-ready instruction |** Create an English-only engineering podcast episode of 42-46 minutes using only the uploaded source pack and this document. Use two hosts: Yael, a young mechanical engineer who asks direct design, commissioning and maintenance questions; and Amir, a senior process and controls engineer who explains software architecture through physical consequences. Give Yael about 45% and Amir about 55% of the speaking time. Make the exchange sound like a post-incident design review, not a lecture or a reading of tables.

Use the CIP-skid power-recovery incident as the continuous story. Do not reveal the complete corrected architecture until the closing. Expand acronyms the first time. Distinguish IEC requirements, PLCopen guidance, ISA/OMAC state-model concepts and vendor tool examples. State clearly that IEC 61131-3 does not guarantee source-code portability between vendors. Do not read long code listings, URLs or standard clause numbers aloud.

Cover functional requirements, invariants, restart behavior, programs/functions/function blocks, LD, ST, FBD, SFC and explicit state machines. Explain single-writer output ownership, layered I/O/device/equipment/sequence/supervisory architecture, typed command-status interfaces, recipe validation, retained data, reusable libraries and restrained object orientation. Include version control, code review, static analysis, simulation, unit/module/sequence/integration tests, release evidence and restore testing. Challenge myths such as “Ladder is always safest,” “Structured Text is just normal software,” “retentive means resume,” and “a backup is version control.” End with the corrected skid architecture, six memorable rules and a transition to Episode 8 on HMI and alarm design.

## 7. Full episode script

### 00:00-03:30 | Cold open - the pump that remembered the wrong thing

**Amir:** A clean-in-place skid is upgraded from one product to three recipes. The software change looks modest: new timing values, a few valve selections and a different rinse sequence. During the first power-recovery test, the circulation pump starts for less than a second while the outlet valve is still closed. The drive trips on pressure, the mechanical seal survives, and the team calls it a commissioning glitch.

**Yael:** Less than a second is long enough to be a real physical event. What did the program do?

**Amir:** Three different routines could write the pump command. The sequence step number was retentive. One ladder rung interpreted that retained step as circulate, another routine was still rebuilding valve status, and the recipe manager had not yet confirmed that the loaded data were valid. Every individual rung looked reasonable when viewed alone.

**Yael:** So the failure was not a syntax error. It was an architecture error.

**Amir:** Exactly. The controller executed valid instructions in a bad ownership model. The network from Episode 6 delivered every bit on time. The software had no single explicit answer to four questions: who owns the output, what state is the equipment in, which data are valid after restart, and what conditions must remain true before energy is released.

**Yael:** Today we are not asking which language is best in the abstract. We are asking how Ladder, Structured Text, Function Block Diagram and Sequential Function Chart can be combined into code that another engineer can understand, test and safely change five years later.

**Amir:** And we will return to the skid at the end with a design that makes the wrong command difficult to express, not merely easy to comment.

### 03:30-07:30 | Requirements before rungs

**Yael:** Automation projects often begin with an I/O list and a programmer opening the engineering tool. What must exist before coding?

**Amir:** A behavioral model. Define operating modes, equipment states, commands, permissives, interlocks, trips, reset rules, restart behavior, manual functions and the boundary to safety systems. For a process unit, add the intended sequence, hold points, abnormal transitions and recipe ownership. For a machine, add cycle states, recovery paths and what the operator may request in each mode.

**Yael:** That sounds like functional design, not software design.

**Amir:** They are inseparable. Software cannot compensate for an undefined process response. Consider the sentence start the pump when ready. Ready must become an explicit expression: correct valve path proven, minimum level, no trip active, drive available, mode allows automatic start, recipe validated, no maintenance lockout, and any required safety function healthy. Each condition needs a source, quality and failure response.

**Yael:** Should we distinguish permissive, interlock and trip?

**Amir:** Yes, even if a company uses different words. A permissive allows a start. An interlock may prevent or remove a command when a required condition is absent. A trip is usually a latched protective response requiring defined reset conditions. The vocabulary must be documented because HMI messages, cause-and-effect tables and maintenance procedures depend on it.

**Yael:** What is an invariant in this context?

**Amir:** A condition that must always be true while a state or action is active. For circulation, an invariant could be: if pump command is on, an open flow path must be proven or a bounded transition timer must be running. Invariants guide code reviews and tests because they turn vague intent into statements that can be challenged.

**Yael:** And restart is a requirement, not a default memory setting.

**Amir:** Precisely. Decide whether the machine returns to stopped, holds its last state, resumes a validated step, or requires operator recovery. Retentive memory is not a continuation strategy. It is only a storage mechanism.

### 07:30-12:30 | IEC 61131-3 in 2025 - a common language, not identical tools

**Yael:** Let us establish the standard landscape. What does the current edition of IEC 61131-3 actually define?

**Amir:** The fourth edition, published in 2025, specifies the syntax and semantics of a unified suite consisting of Structured Text, Ladder Diagram and Function Block Diagram. It also defines Sequential Function Chart elements for structuring programs and function blocks, plus configuration elements. Instruction List, which older engineers may know from legacy systems, is not part of that current language suite.

**Yael:** Does compliance mean I can move a project from any vendor to another vendor and compile it?

**Amir:** No. The standard improves common concepts and language semantics, but tools differ in supported subsets, libraries, data types, task models, hardware configuration, motion, safety, visualization, diagnostics and vendor extensions. Even when syntax imports successfully, behavior and lifecycle evidence still require verification.

**Yael:** What are the common building blocks beyond the visible language?

**Amir:** Data types, variables, configurations, resources, tasks and Program Organization Units. POUs include programs, function blocks and functions. A function should behave like a calculation without persistent instance state. A function block has instance memory and suits a valve, motor, controller or reusable stateful behavior. A program coordinates application behavior and is scheduled by the runtime.

**Yael:** And IEC TR 61131-8?

**Amir:** It provides application and implementation guidance for the languages. IEC 61131-10 defines an XML exchange format for IEC 61131-3 projects. That supports tool exchange and version-control workflows, but it does not erase vendor-specific semantics or guarantee one-click migration.

**Yael:** The 2025 edition also mentions UTF-8 strings.

**Amir:** Yes, and that reminds us that controller software handles richer data. But richer language features should not tempt us to place every IT-style function inside the real-time control task. Architecture still separates deterministic control from reporting, analytics and noncritical services.

**Yael:** Then standardize the mental model, and verify the implementation.

**Amir:** Exactly. Use the standard as a common engineering grammar, not as a substitute for platform documentation and system testing.

## 12:30-17:30 | Ladder Diagram - visible power flow and invisible traps

**Yael:** Ladder remains the language many electricians and maintenance technicians prefer. What is it genuinely good at?

**Amir:** Boolean logic that resembles relay circuits: start-stop circuits, permissive chains, interlocks, mode selection, alarms and simple device control. The visual path can make an open condition easy to locate online. That works when the architecture is disciplined.

**Yael:** The phrase when the architecture is disciplined sounds important.

**Amir:** Very. Ladder becomes difficult when one coil is written in many places, when latches are used without explicit reset ownership, when long branches hide priority, or when sequence state is represented by dozens of unrelated bits. The screen may look familiar while the program has no clear behavioral model.

**Yael:** Explain the multiple-writer problem using our pump.

**Amir:** Suppose Manual, Automatic Sequence and Recovery each contain a rung that writes PumpCmd. The final value may depend on scan order or platform rules. A change in one routine can silently override another. A better design lets each layer issue a request to one pump module. That module alone owns the physical command and resolves mode, permissive, interlock and feedback.

**Yael:** What about set and reset coils?

**Amir:** They are not forbidden, but every latched state needs a defined owner, reset path, initialization rule and diagnostic meaning. A latched bit that survives longer than its requirement becomes hidden memory. The software should reveal a state or command source, not force someone to search for the last rung that set a coil.

**Yael:** Online monitoring can also create false confidence.

**Amir:** Yes. A highlighted rung shows current evaluation, not the history that produced a retained bit or timing between tasks. It may also hide a physical output fault. Distinguish requested command, controller output, field power, actuator state and process effect.

**Yael:** Ladder is strongest for transparent decisions, not for storing the entire plant history in contacts.

**Amir:** Well put. Keep rungs short enough to explain, name conditions by engineering meaning, and make output ownership obvious.

## 17:30-22:30 | Structured Text - expressive power with real-time responsibilities

**Yael:** Structured Text looks familiar to engineers who know Pascal, C or Python. Where does it earn its place?

**Amir:** Calculations, arrays, loops, data conversion, recipe validation, string handling, algorithms, state machines and reusable libraries. It expresses complex transformations more compactly than a graphical network. A pressure-compensation calculation with ten intermediate values may be clearer as typed expressions than as a wall of blocks.

**Yael:** What are the common mistakes?

**Amir:** Dense code that only the author can read, implicit type conversions, hidden state changes inside functions, loops over variable-sized data without considering worst-case execution time, array indexing without range protection, mixed engineering units and desktop-style assumptions about exceptions.

**Yael:** Give an example of bounded execution.

**Amir:** If a periodic task has a ten-millisecond budget, a loop that processes every recipe item must have a known maximum. A search that is harmless with twenty records may overrun when configuration grows to two thousand. Real-time reviews ask not only whether the result is correct, but what the worst execution path is and what happens if input data are malformed.

**Yael:** How should units appear in code?

**Amir:** Prefer strongly typed structures, meaningful names and explicit conversions. `Temperature_C`, `Pressure_bar_g` and `Duration_ms` are more informative than `Value1`, even if the platform lacks true unit types. Centralize scaling at the I/O boundary so internal logic operates in engineering units. Do not scatter raw counts and magic constants.

**Yael:** Is a CASE statement a good way to build a state machine?

**Amir:** It can be excellent if states are enumerated, transitions are explicit, entry actions are controlled, illegal states are handled and outputs are derived consistently. It becomes poor when every state writes many devices directly and transition conditions are duplicated. The state machine should coordinate modules, not bypass them.

**Yael:** And comments?

**Amir:** Comments should explain why, constraints and assumptions. The code should reveal what. A comment saying start pump above M120 adds nothing. A comment explaining that a three-second valve-proof delay is based on a validated mechanical travel envelope is useful and should point to the controlled requirement.

## 22:30-26:30 | Function Block Diagram - signal flow, control and execution order

**Yael:** Function Block Diagram often appears in process control. What makes it suitable?

**Amir:** It represents signal flow and relationships between functions. PID control, filtering, scaling, voting, analog conditioning and control-valve logic can be easy to review as connected blocks. A mechanical engineer can follow the path from measured variable through validation and controller output to the final element.

**Yael:** Does the drawing guarantee a left-to-right execution order?

**Amir:** Do not assume that from appearance alone. Platforms define network order, block execution and feedback handling. Cyclic dependencies need care. The engineer must understand the tool's execution model and make dependencies explicit where order matters.



**Yael:** What causes FBD to become unreadable?

**Amir:** Huge sheets, crossing lines, implicit global variables, mixed abstraction levels and blocks with unclear side effects. A page containing sensor scaling, mode logic, PID tuning, alarm latching and valve-output ownership is not a control diagram; it is an architecture collapse.

**Yael:** Then one network should tell one story.

**Amir:** Yes. Keep the signal path visible, encapsulate repeated behavior in function blocks, expose status and quality, and separate basic control from sequence coordination. FBD is powerful when each block has a clear contract.

**Yael:** Would you implement every PID loop in a custom block?

**Amir:** Usually not. Use proven platform or company libraries, but wrap them where necessary to standardize units, modes, tracking, output limits, bumpless transfer, alarms and diagnostics. Reuse should reduce variation, not hide critical behavior.

## 26:30-31:30 | SFC and state machines - make sequence state explicit

**Yael:** Sequential Function Chart is sometimes called a programming language and sometimes not. Which is correct?

**Amir:** In IEC 61131-3:2025, SFC is a set of elements used to structure the internal organization of programs and function blocks. Steps hold state, transitions define when execution moves, and actions use other language elements. PLCopen emphasizes that SFC is especially useful for sequential processes and state-machine behavior.

**Yael:** What does explicit state buy us?

**Amir:** Observability and controlled transitions. Instead of guessing which combination of bits means rinsing, the program states Rinsing. From that state, only defined transitions are legal: complete, hold, abort or fault. Entry and exit behavior can be reviewed, displayed and tested.

**Yael:** How detailed should the states be?

**Amir:** Detailed enough to represent meaningful equipment behavior, not every scan action. A state should have a purpose, entry criteria, active invariants, completion criteria, timeout and abnormal exits. If a state exists only to wait one scan, it probably models implementation detail rather than process behavior.

**Yael:** What about parallel branches?

**Amir:** They can represent concurrent activities, but synchronization must be explicit. If two branches control shared equipment, ownership and completion rules must be clear. Elegant-looking parallelism can create a race when one branch faults and the other continues.

**Yael:** PackML is often mentioned with state models.

**Amir:** OMAC PackML and ISA TR88 machine-state work provide common concepts such as stopped, starting, execute, holding, held, aborting and aborted. They can improve consistency across machines, but the model must be mapped to the actual process and not pasted as a decorative layer over unrelated logic.

**Yael:** For our skid, would recipe steps and unit mode be the same state machine?

**Amir:** Usually separate them. A unit mode such as Off, Manual, Automatic or Maintenance governs permissions. The sequence state describes where the recipe is. Device modules retain their own states. Layered state machines prevent one giant chart from owning every detail.

## 31:30-36:30 | A maintainable automation architecture

**Yael:** Let us build the layers from the terminals upward.



**Amir:** First, I/O abstraction. Map raw channels, scaling, inversion, quality and simulation at one boundary. Second, device modules for pumps, valves, heaters, drives and instruments. Each module owns commands, permissives, interlocks, feedback, faults and diagnostics for one device or tightly defined assembly.

**Yael:** Third would be equipment modules?

**Amir:** Yes. An equipment module coordinates devices to provide a capability: circulation path, dosing, heating, positioning or transfer. It requests device actions and exposes compact status. Above that, sequence or phase logic coordinates capabilities to execute a recipe or machine cycle. A supervisory layer manages modes, production orders, handshakes and HMI-facing summaries.

**Yael:** Where does a PID loop belong?

**Amir:** Usually in the equipment or basic-control layer, close to the process variable and final element. The sequence requests a setpoint and mode; it should not reimplement the controller. This lets the loop remain in manual or tracking during sequence transitions without scattering logic across steps.

**Yael:** You keep returning to ownership.

**Amir:** Because ownership prevents contradictory commands. One module writes each physical output. Higher layers send requests through defined interfaces. A pump module may receive `AutoRequest`, `ManualRequest` and `StopRequest`, but it resolves them with documented priority and exposes `CommandSource`, `Ready`, `Running`, `Faulted` and `NotPermittedReason`.

**Yael:** That also improves HMI design later.

**Amir:** Exactly. The interface can display why a start is blocked without reconstructing logic from dozens of tags. It also supports simulation: replace the I/O boundary with a model while preserving module interfaces.

**Yael:** What should cross module boundaries?

**Amir:** Commands, status, configuration and events with explicit types. Avoid unrestricted access to internal variables. A module contract states what the caller may request, what feedback means, what timing assumptions exist and how faults are acknowledged.

## 36:30-40:30 | Data models, reuse and object orientation without architecture theatre

**Yael:** Reusable code sounds obviously good. Why does it sometimes make projects harder?

**Amir:** Because copied code is called a library, or because a generic block exposes fifty parameters to cover every imaginable machine. Reuse works when behavior is stable, interfaces are small, documentation is controlled, versions are traceable and the consuming project can test the exact library release.

**Yael:** Start with data types.

**Amir:** Use enumerations for modes and states, structures for device commands and status, and named constants for limits. A recipe should be a validated structure with a version and checksum or equivalent integrity method where appropriate, not a loose set of tags. Separate configuration from live state and operator-entered values from approved values used by control.

**Yael:** Function blocks give us instances. What does object-oriented programming add?

**Amir:** Depending on the platform, methods, interfaces, inheritance or properties can improve encapsulation and polymorphism. PLCopen has guidance for object orientation in industrial control. But OOP is a tool, not a maturity badge. Deep inheritance, dynamic relationships and hidden side effects can make online troubleshooting worse.

**Yael:** When is an interface useful?

**Amir:** When different implementations must present the same contract. A simulated valve and a physical valve can implement the same command-status interface. Different drive families can expose a common

equipment-facing API while vendor-specific diagnostics remain inside adapters. This supports substitution without pretending the hardware is identical.

**Yael:** How should library versions be managed?

**Amir:** Semantic versioning can help, but the essential rules are controlled release, dependency recording, backward-compatibility policy, migration notes and regression tests. A machine project should know exactly which library version produced the downloaded binary. Never update a global library silently across commissioned projects.

**Yael:** And avoid premature generalization.

**Amir:** Yes. First make one design correct and observable. Generalize after comparing real variants. The best reusable block standardizes dangerous and repetitive parts while leaving application-specific process logic visible.

## 40:30-44:00 | Version control, static analysis and tests that survive commissioning

**Yael:** Automation teams often say the PLC project is backed up. Is that version control?

**Amir:** No. A backup preserves a file at one moment. Version control records intentional changes, authorship, review, release tags and the relationship between branches. It should let the team answer what changed, why, who approved it, which controller received it and how to reproduce the released build.

**Yael:** Graphical languages make diffs harder.

**Amir:** They can. Use vendor source-control integrations, textual exchange formats where reliable, or controlled export formats such as PLCopen XML. But never treat a successful textual merge as proof that the control behavior is correct. Compile, analyze and test the resulting project in the target engineering environment.

**Yael:** What does static analysis contribute?

**Amir:** It finds patterns before runtime: unused variables, implicit conversions, unreachable code, naming violations, excessive complexity, suspicious assignments, multiple writes or library-rule violations, depending on the tool. CODESYS, for example, provides rule- and metric-based static analysis. Similar capabilities exist in other ecosystems. The rule set should be tailored and deviations reviewed rather than blindly suppressed.

**Yael:** Then tests. What is a useful hierarchy?

**Amir:** Unit tests for functions and function blocks; module tests with simulated I/O; sequence tests covering normal, hold, abort and recovery paths; integration tests with real drives, networks and instruments; and site acceptance tests under actual process constraints. Simulation is excellent for breadth, but it cannot reproduce every physical timing, wiring fault or vendor-firmware behavior.

**Yael:** Our power-loss incident should become a test family.

**Amir:** Exactly. Remove power or simulate restart in every meaningful sequence state. Vary whether valve feedback is stale, the recipe is invalid, the drive is unavailable or the operator requests manual mode. Verify that outputs remain safe, the state becomes RecoveryRequired, the reason is visible and resumption is impossible until the defined recovery procedure succeeds.

**Yael:** And release evidence?

**Amir:** Archive source revision, tool and compiler version, target firmware, libraries, configuration, test report, approved change request, checksum or signature where supported, and a tested restore package. A backup that has never been restored is only hope stored on a server.

**Yael:** Online edits?

**Amir:** They may be necessary during commissioning, but each one is a controlled change. Record it, reconcile it into the master source, rerun the affected tests and create a new release. The controller should never become the only place where the truth exists.

## 44:00-46:00 | Corrected architecture and closing

**Yael:** Let us resolve the skid. What changed?

**Amir:** The physical pump output is now written only by PumpModule. Manual control, the automatic sequence and recovery logic send requests; they cannot write the output directly. The module resolves priority, proves the valve path, verifies level and drive readiness, applies the stop or trip conditions and exposes one reason when starting is blocked.

**Yael:** The retained step number?

**Amir:** Retained only as production evidence, not as an instruction to resume. After uncontrolled power loss, the unit enters RecoveryRequired. The recipe is reloaded and validated, live device state is rebuilt from feedback, and the operator follows an explicit recovery sequence. Automatic circulation cannot resume until the equipment state and process conditions are reconciled.

**Yael:** The code was also split into I/O, device, equipment, sequence and supervisory layers, with enumerated states and typed interfaces.

**Amir:** And the team added static-analysis rules for multiple writers, unit tests for PumpModule, sequence simulations for every abnormal exit and hardware tests for power recovery. The fix is not one extra normally closed contact. It is a software structure that makes ownership and state visible.

**Yael:** Give us the final rules.

**Amir:** Choose the language that makes the engineering intent easiest to review. Give every output one owner. Model modes and sequence states explicitly. Treat retained data and restart as separate design decisions. Keep module interfaces typed and small. Control libraries and releases. Test failures and recovery, not only the happy path.

**Yael:** In Episode 8 we will use those interfaces to build the operator's window into the machine: HMI hierarchy, high-performance graphics, navigation, commands, alarms and the difference between a screen that looks impressive and one that supports a correct decision.

**Amir:** Good control software knows what the machine is doing. A good HMI helps a human understand why.

## 8. Producer and host notes

- Keep the physical consequence of every software concept visible: pressure, motion, heat, flow, energy and operator action.
- Do not stage LD versus ST as a cultural argument. Show where each representation improves or damages reviewability.
- Use the terms permissive, interlock and trip consistently, while acknowledging that company definitions can differ.
- Do not imply that IEC 61131-3 compliance creates identical vendor behavior or automatic portability.
- Explain one-writer ownership several times using the pump output, but avoid repetitive wording.
- Never suggest that simulation or unit testing replaces commissioning with real I/O, drives, networks and process constraints.
- Do not read source URLs or dense checklists aloud. Use them as production background.
- Keep the complete root cause and corrected recovery flow for the closing segment.

## 9. Engineering checklist for PLC software

1. Are operating modes, equipment states, commands, authority and transitions explicitly defined?

2. For every energized action, what permissives, interlocks, trips and invariants apply?
3. What is the safe and intended behavior after warm restart, cold restart, power loss and communications recovery?
4. Does each physical output have exactly one software owner?
5. Are commands, actual state, process effect, signal quality and command source separated?
6. Is the selected language the clearest representation for the engineering behavior being implemented?
7. Are state machines enumerated, legal transitions explicit and abnormal exits complete?
8. Are task period, worst-case execution time, loops, data size and shared-data interactions bounded?
9. Are scaling and units centralized and free of unexplained raw counts or magic constants?
10. Are recipe/configuration data validated, versioned and separated from live and retained control state?
11. Do module interfaces expose compact typed command, status, reason and diagnostic structures?
12. Are library dependencies pinned, documented, tested and recoverable for the commissioned release?
13. Does version control preserve meaningful change history, review, release tags and reproducible exports?
14. Are static-analysis findings reviewed and exceptions justified rather than globally suppressed?
15. Do tests cover start, stop, hold, abort, timeout, stale feedback, invalid data, power loss and recovery?
16. Can the released source, toolchain, libraries, firmware, configuration and tested backup restore the system?

## 10. Glossary

| Term            | Meaning in this episode                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------|
| IEC 61131-3     | IEC standard defining programming-language syntax and semantics for programmable controllers. |
| POU             | Program Organization Unit: program, function block or function.                               |
| LD              | Ladder Diagram, a graphical language based on relay-logic concepts.                           |
| ST              | Structured Text, a high-level textual IEC 61131-3 language.                                   |
| FBD             | Function Block Diagram, a graphical representation of functions and signal flow.              |
| SFC             | Sequential Function Chart elements for organizing steps, transitions and actions.             |
| State machine   | Explicit model of states, permitted transitions and behavior in each state.                   |
| Single writer   | Architecture in which one module alone resolves and writes a physical output.                 |
| Retentive data  | Data configured to survive defined restart or power-loss conditions.                          |
| Invariant       | Condition that must remain true while a state or action is active.                            |
| Static analysis | Source-code evaluation using rules or metrics without executing the application.              |
| Regression test | Test confirming that existing behavior remains correct after a change.                        |

| Term        | Meaning in this episode                                           |
|-------------|-------------------------------------------------------------------|
| PLCopen XML | IEC 61131-10 exchange format for IEC 61131-3 project information. |
| PackML      | OMAC machine-state and data concepts based on ISA-88 terminology. |

## 11. Standards and source map

Primary and official sources used to structure this episode. Purchase or contractual access may be required for complete standards. Verify the adopted edition, vendor version and project requirements.

IEC 61131-3:2025, Programmable controllers - Part 3: Programming languages, edition 4.0 - <https://webstore.iec.ch/en/publication/68533>

IEC TR 61131-8:2017, Guidelines for the application and implementation of programming languages - <https://webstore.iec.ch/en/publication/33021>

IEC 61131-10:2019, PLC open XML exchange format - <https://webstore.iec.ch/en/publication/33034>

PLCopen, Software Construction Guidelines: coding, compliant libraries, SFC, OOP and quality metrics - <https://www.plcopen.org/guidelines/software-construction-guidelines/>

PLCopen, Structuring with Sequential Function Chart - [https://www.plcopen.org/application/files/8717/3868/2051/plcopen\\_structuring\\_with\\_sfc.pdf](https://www.plcopen.org/application/files/8717/3868/2051/plcopen_structuring_with_sfc.pdf)

OMAC, PackML - <https://www.omac.org/packml>

ISA, TR88.00.02 Machine and Unit States preview - [https://www.isa.org/getmedia/300dbd50-d549-41ac-b372-a5e52f32fc97/tr\\_880002\\_preview.pdf](https://www.isa.org/getmedia/300dbd50-d549-41ac-b372-a5e52f32fc97/tr_880002_preview.pdf)

CODESYS Git documentation - <https://content.helpme-codesys.com/en/CODESYS%20Git/index.html>

CODESYS Static Analysis documentation - [https://content.helpme-codesys.com/en/CODESYS%20Static%20Analysis/\\_san\\_start\\_page.html](https://content.helpme-codesys.com/en/CODESYS%20Static%20Analysis/_san_start_page.html)

CODESYS Test Manager documentation - [https://content.helpme-codesys.com/en/CODESYS%20Test%20Manager/\\_tm\\_start\\_page.html](https://content.helpme-codesys.com/en/CODESYS%20Test%20Manager/_tm_start_page.html)

Siemens, SIMATIC STEP 7 / TIA Portal and TIA Portal Test Suite documentation - <https://www.siemens.com/en-gb/products/tia-portal/step7/>

Rockwell Automation, Studio 5000 Logix Designer import and export documentation - <https://www.rockwellautomation.com/en-us/docs/studio-5000-logix-designer/>

**How to use the source map |** Use standards to establish common language and requirements, PLCopen and ISA/OMAC material for construction patterns, and exact vendor manuals for task scheduling, language subsets, source control, online change, testing and target behavior. Never infer safety certification or portability from a generic language example.

## 12. Quality gate before recording

- The script sustains approximately 42-46 minutes at a natural two-host technical pace.
- The current IEC 61131-3:2025 language suite is described without presenting Instruction List as current.
- No vendor-neutral portability, certification or clause-level compliance is invented.
- The core incident is resolved through ownership, state and recovery architecture rather than a cosmetic rung patch.
- LD, ST, FBD and SFC receive distinct strengths, risks and review questions.
- Retention, restart, recipe validation and live equipment state are kept as separate concepts.
- Testing includes failure and recovery paths plus real-hardware integration limits.
- The close creates a clean transition to HMI and alarm design.